

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

ROHIT L - 192511236

QUESTIONS

Annexure A Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug?
Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent?
Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.
The delivery network is represented as a weighted graph:

Artificial Intelligence & Problem Solving Solutions

Comprehensive Solutions to Classical AI State-Space Search, Agent Architectures & Pathfinding

1. The Water Jug Problem (4-Gallon & 3-Gallon Jugs)

Problem Description: You are given two jugs—a 4-gallon jug and a 3-gallon jug—with no measuring marks. A pump is available to fill the jugs, and water can be poured onto the ground or between the jugs. The objective is to obtain exactly **2 gallons of water** in the 4-gallon jug.

Formal State-Space Formulation

- **State Representation:** An ordered pair (x, y) where $x \in \{0, 1, 2, 3, 4\}$ represents gallons in the 4-gallon jug, and $y \in \{0, 1, 2, 3\}$ represents gallons in the 3-gallon jug.
- **Start State:** $(0, 0)$ — Both jugs are empty.
- **Goal State:** $(2, y)$ for any $y \in \{0, 1, 2, 3\}$ — Exactly 2 gallons in the 4-gallon jug.

Production Rules / State Transition Operators

Rule	Condition	Action / Operator	Resulting State
1	$x < 4$	Fill 4-gallon jug	$(4, y)$
2	$y < 3$	Fill 3-gallon jug	$(x, 3)$
3	$x > 0$	Empty 4-gallon jug on ground	$(0, y)$
4	$y > 0$	Empty 3-gallon jug on ground	$(x, 0)$
5	$x + y \geq 4$ and $y > 0$	Pour 3-gal into 4-gal until full	$(4, y - (4 - x))$
6	$x + y \geq 3$ and $x > 0$	Pour 4-gal into 3-gal until full	$(x - (3 - y), 3)$
7	$x + y \leq 4$ and $y > 0$	Pour all water from 3-gal into 4-gal	$(x + y, 0)$
8	$x + y \leq 3$ and $x > 0$	Pour all water from 4-gal into 3-gal	$(0, x + y)$

Optimal Step-by-Step Solution Sequence

Step	4-Gal (x)	3-Gal (y)	Rule Applied	Description
0	0	0	Initial State	Both jugs are completely empty.
1	0	3	Rule 2	Fill the 3-gallon jug completely from the pump.
2	3	0	Rule 7	Pour all water from 3-gal jug into 4-gal jug.

3	3	3	Rule 2	Fill the 3-gallon jug completely again from the pump.
4	4	2	Rule 5	Pour from 3-gal into 4-gal until 4-gal is full (1 gal transferred).
5	0	2	Rule 3	Empty the 4-gallon jug onto the ground.
6	2	0	Rule 7	Pour remaining 2 gallons from 3-gal jug into 4-gal jug. (Goal Met)

Alternative Solution Sequence (8 Steps): Fill 4-gal (4,0) → Pour to 3-gal (1,3) → Empty 3-gal (1,0) → Transfer 4-gal to 3-gal (0,1) → Fill 4-gal (4,1) → Pour 4-gal to 3-gal (2,3).

2. Mars Rover Exploration Agent Analysis

The Mars Rover is an autonomous robotic agent designed for surface navigation, geological sample analysis, and astrobiological exploration in extreme planetary environments.

i. Agent Percepts

- **Visual & Spatial:** High-resolution stereoscopic navigation cameras (Navcams), hazard avoidance cameras (Hazcams), and LiDAR/optical terrain sensors.
- **Spectrometric & Chemical:** Alpha Particle X-ray Spectrometer (APXS), laser-induced breakdown spectroscopy (ChemCam/SuperCam), infrared and Raman spectrometers.
- **Mechanical & Physical:** Wheel torque sensors, wheel speed encoders, 3-axis accelerometers, gyroscopes, and inertial measurement units (IMU) for slip detection.
- **Environmental & Atmospheric:** Temperature, atmospheric pressure, wind velocity, solar radiation sensors, and dust deposition monitors.
- **System Health:** Internal temperature, solar panel voltage / MMRTG power output, battery state-of-charge, arm force/torque sensors.

ii. Characterization of the Operating Environment

Property	Classification	Detailed Rationale
Observability	Partially Observable	Sensors cannot perceive underground geology, hidden obstacles behind terrain, or dust storms in advance.
Determinism	Stochastic	Actions have uncertain outcomes; wheel slipping on loose regolith or wind gusts create non-deterministic results.

Episodic vs. Sequential	Sequential	Current navigation and sampling decisions directly dictate future position, power availability, and safety.
Static vs. Dynamic	Dynamic	The environment changes continuously while the rover deliberates (e.g., changing sunlight, moving shadows, wind).
Discrete vs. Continuous	Continuous	Velocity, steering angle, joint angles, power consumption, time, and spatial coordinates are continuous variables.
Agent Type	Single / Semi-Autonomous	Operates as a single physical entity on Mars, coordinating asynchronously with Earth ground control.

iii. Action Space (Agent Actuators)

- **Mobility:** Individual wheel drive motors, corner steering actuators, rocker-bogie suspension adjustments, speed/heading changes.
- **Manipulation & Drilling:** Robotic arm joint articulation, rotary-percussive drill deployment, coring, sample caching mechanism, dust removal tool.
- **Scientific Operations:** Laser firing (ChemCam), instrument deployment, sample heating/gas analysis, camera positioning and exposure control.
- **Communication & Power:** High-Gain/Ultra-High-Frequency (UHF) antenna pointing, data transmission to orbiters, power distribution/sleep state management.

iv. Performance Evaluation Metrics

- **Scientific Yield:** Quantity, diversity, and purity of geological/atmospheric samples collected and successfully analyzed/cached.
- **Safety & Survival Rate:** Zero catastrophic incidents (e.g., getting trapped in sand dunes, tipping over, or colliding with rocks).
- **Energy Efficiency:** Ratio of meters traversed and science performed per watt-hour consumed.
- **Autonomous Path Accuracy:** Minimization of path deviation, wheel slip error, and localization uncertainty during long autonomous drives.

v. Recommended Agent Architecture & Justification

Selected Architecture: Hybrid Deliberative / Reactive Architecture (3-Tiered Architecture)

- **Reactive Layer (Lower Tier):** Provides immediate reflex responses (sub-second latency) for critical safety, such as halting wheels upon detecting severe slip, obstacle proximity, or excessive tilt angle.
- **Executive Layer (Middle Tier):** Coordinates plan execution, monitors task status, handles local re-routing, and manages resource thresholds.
- **Deliberative Layer (Upper Tier):** Handles complex problem solving, high-level path planning (e.g., A* over terrain mesh), path optimization, and science target scheduling.

3. 8-Queens Problem Formulation & Search Space Exploration

The 8-Queens problem requires placing eight chess queens on an 8×8 chessboard such that no two queens attack each other (i.e., no two queens share the same row, column, or diagonal).

Problem Component Formulation

- **Incremental State Representation:** A vector $S = (q_1, q_2, \dots, q_k)$ where $k \in \{0, 1, \dots, 8\}$. Here, $q_i \in \{1, \dots, 8\}$ represents the row position of the queen in column i . (Example invalid state given in problem: Queen in Col 1 and Col 8 share the same diagonal).
- **Initial State:** An empty board represented by an empty tuple $()$ or $k = 0$.
- **Actions / Successor Function:** Add a queen to any row $r \in \{1, \dots, 8\}$ in the leftmost empty column $k + 1$ such that it does not conflict with already placed queens.
- **Transition Model:** Returns state $(q_1, \dots, q_k, r)$ after placing a queen at column $k + 1$, row r .
- **Goal Test:** $k = 8$ queens are placed on the board with zero conflicts: $\forall i, j \in \{1, \dots, 8\}, i \neq j \Rightarrow q_i \neq q_j$ and $|q_i - q_j| \neq |i - j|$.
- **Path Cost:** Zero or step cost of 1 per placement (only final state matters).

Search Space Analysis & Reduction

Formulation Type	Search Space Size	Evaluation
Naive Placement	$C(64, 8) = 4.42 \times 10^9$	Extremely large; allows multiple queens per cell/column.
One Queen Per Column	$8^8 = 16,777,216$	Restricts placement to 1 queen per column.
Permutation Formulation	$8! = 40,320$	Ensures distinct rows and columns; checks diagonal constraints only.
Backtracking with CSP	$< 2,000$ states explored	Prunes invalid subtrees immediately upon constraint violation.

Comparison of Search Strategies for 8-Queens

1. Depth-First Backtracking Search: Explores column by column. If placing a queen in column k causes a conflict with existing queens, the search backtracks to column $k - 1$ and tries the next row. It is complete and memory efficient ($O(d)$ space).

2. Local Search (Hill-Climbing / Min-Conflicts): Starts with all 8 queens on the board (1 per column). Heuristic $h(n)$ = number of attacking pairs. The algorithm minimizes $h(n)$ by moving a queen in a column to a row with minimum conflicts. Solves the problem in $O(1)$ average time.

Constraint Satisfaction Problem (CSP) Representation

- **Variables:** $X = \{Q_1, Q_2, Q_3, Q_4, Q_5, Q_6, Q_7, Q_8\}$
- **Domains:** $D_i = \{1, 2, 3, 4, 5, 6, 7, 8\}$ for all $i \in \{1, \dots, 8\}$
- **Constraints:** Non-attacking condition: $Q_i \neq Q_j$ and $|Q_i - Q_j| \neq |i - j|$ for all $i \neq j$.

4. OLA Cab Booking Problem-Solving Agent

Agent Identification & Classification

The OLA Cab Booking system is best modeled as a Utility-Based Problem-Solving Agent that incorporates multi-criteria search and planning.

Rationale: Simple goal-based agents only look for any route/cab to destination. However, cab selection involves trade-offs across multiple competing factors: trip cost, estimated time of arrival (ETA), vehicle comfort/category (Mini, Micro, Sedan, Shared, Prime), driver rating, and surge pricing. A utility function maps these attributes into a real number allowing the agent to select the overall best option for the user.

PEAS Breakdown for OLA Agent

- **Performance Measure:** User satisfaction (high comfort, low cost, minimal wait time), trip safety, driver match efficiency, system throughput.
- **Environment:** Dynamic urban road network, real-time traffic conditions, variable weather, active drivers and competing customer requests.
- **Actuators:** Mobile app UI display, booking request dispatcher, payment processing API, notification system.
- **Sensors:** GPS location sensor, user selection inputs, driver telemetry/GPS, traffic data feed API.

Pseudocode: OLA Utility-Based Cab Booking Agent

```
CLASS CabOption:
    String category // Mini, Micro, Sedan, Shared, Prime
    Float fareCost
    Float etaMinutes
    Float driverRating
    String cabID

FUNCTION OlaProblemSolvingAgent(CustomerRequest req, List<Driver> activeDrivers, Map Grid):
    // Step 1: Formulate Goal and Filter Feasible Cabs
    Point pickupLoc = req.origin
    Point dropLoc = req.destination
    String preferredCategory = req.userPreference // e.g., "Prime" or "ANY"

    List<CabOption> validOptions = []

    FOR EACH driver IN activeDrivers:
        IF driver.isAvailable AND Distance(driver.location, pickupLoc) < MAX_RADIUS:
            Route pickupRoute = A_Star_Search(Grid, driver.location, pickupLoc)
            Route tripRoute = A_Star_Search(Grid, pickupLoc, dropLoc)

            Float cost = CalculateFare(driver.category, tripRoute.distance,
CurrentTrafficMultiplier())
            Float totalETA = pickupRoute.estimatedTime

            IF preferredCategory == "ANY" OR driver.category == preferredCategory:
                CabOption option = NEW CabOption(driver.category, cost, totalETA, driver.rating,
driver.id)
                validOptions.Append(option)

    // Step 2: Evaluate Utility Function for Each Available Option
    // Utility U = w1*(1/Cost) + w2*(1/ETA) + w3*(Rating) + w4*(ComfortWeight)
    CabOption bestCab = NULL
    Float maxUtility = -INFINITY
```

```

FOR EACH option IN validOptions:
    Float utility = CalculateUtility(option, req.userWeights)
    IF utility > maxUtility:
        maxUtility = utility
        bestCab = option

// Step 3: Execute Action
IF bestCab IS NOT NULL:
    BookingResult result = ConfirmBooking(req.userID, bestCab.cabID)
    RETURN DisplayToUser("Cab Booked: " + bestCab.category + ", Cost: " + bestCab.fareCost)
ELSE:
    RETURN DisplayToUser("No cabs available matching criteria. Try adjusting vehicle type.")

```

5. Logistics Delivery Network & Uniform Cost Search (UCS)

A logistics company operates a warehouse delivery network represented as a weighted graph $G = (V, E)$. The goal is to find the least-cost path from starting warehouse S to destination warehouse G using Uniform Cost Search (UCS).

Logistics Network Graph Representation

Vertices $V = \{S, A, B, C, D, E, G\}$. Directed edges with travel costs $c(u, v)$ (fuel + time):

- $S \rightarrow A: 3, S \rightarrow B: 5, S \rightarrow C: 2$
- $A \rightarrow D: 4, A \rightarrow B: 1$
- $B \rightarrow D: 2, B \rightarrow E: 6$
- $C \rightarrow B: 2, C \rightarrow E: 7$
- $D \rightarrow E: 1, D \rightarrow G: 5$
- $E \rightarrow G: 2$

Step-by-Step Execution Trace of UCS Algorithm

Uniform Cost Search expands the node n with the lowest path cost $g(n)$ from the Priority Queue (PQ).

Step	Pop	$g(n)$	Priority Queue State [Node: $g(n)$, Path]	Explored Set
0	-	0	[S: 0, S]	$\emptyset$
1	S	0	[C: 2, S-C], [A: 3, S-A], [B: 5, S-B]	{S}
2	C	2	[A: 3, S-A], [B: 4, S-C-B], [B: 5, S-B], [E: 9, S-C-E]	{S, C}
3	A	3	[B: 4, S-C-B], [B: 4, S-A-B], [D: 7, S-A-D], [E: 9, S-C-E]	{S, C, A}
4	B	4	[D: 6, S-C-B-D], [D: 7, S-A-D], [E: 9, S-C-E], [E: 10, S-C-B-E]	{S, C, A, B}
5	D	6	[E: 7, S-C-B-D-E], [E: 9, S-C-E], [G: 11, S-C-B-D-G]	{S, C, A, B, D}

6	E	7	[G: 9, S-C-B-D-E-G], [G: 11, S-C-B-D-G]	{S, C, A, B, D, E}
7	G	9	Goal Reached! Search Terminates.	{S, C, A, B, D, E, G}

Optimal Solution Path Summary

Optimal Path: $S \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow G$
 Total Minimum Travel Cost: $2 + 2 + 2 + 1 + 2 = 9$

Optimality & Algorithm Comparison

- Guarantee of Optimality:** UCS is guaranteed to find the optimal path because step costs are strictly positive ($c \geq \epsilon > 0$) and nodes are expanded in monotonic order of path cost $g(n)$.
- Comparison with BFS:** BFS only finds the path with the fewest hops ($S \rightarrow A \rightarrow D \rightarrow G$ cost = 12 or $S \rightarrow B \rightarrow D \rightarrow G$ cost = 12), which is sub-optimal compared to UCS cost 9.